

1. Что реализовано

1. Был добавлен пакет `coinflipper` с структурой `CoinFlipper` и операциями `New`, `Flip`, `GetSeed`
2. Был изменён пакет `skiplist`:
 - был создан файл `helpers.go` с функциями сравнения ключей и копирования байтов
 - был создан файл `node.go` с объявлением структуры узла `SkipList` и функциями создания и манипулирования узлов
 - в файл `skiplist.go` были реализованы операции `New`, `Put`, `Get`, `Delete`, `Scan` и добавлены новые:
 - `SetProbability` - установить вероятность перехода узла на более высокий уровень
 - `IsEmpty` - является ли `SkipList` пустым
 - `GetLevelsNumber` - получить количество уровней в `SkipList`
 - `GetRepresentation` - получить строковое представление `SkipList`
 - был создан файл `iterator.go` с объявлением итератора для `SkipList` и операциями `Next` и `Close` над итератором

2. Инварианты и семантика

- https://github.com/ktblsva/streaming_algo_course/blob/master/docs/go_intro/10_skiplist_and_iterators.md#1-семантика-scanstartend-самая-частая-причина-падений
- `Put` обновляет значение, если ключ уже есть.

3. Принятые решения

1. Максимальное количество уровней структуры не задано и никак не ограничено. Сделано это так как количество ключей структуры может быть заранее неизвестно, поэтому передавать в конструкторе максимальный уровень или фиксировать его в константе нецелесообразно, так как в таком случае средние сложности операций будут хуже, чем $O(\log N)$. То что из-за рандома при "небольшом" количестве ключей количество уровней будет "большим" - маловероятно.
2. Случайность используется в виде "flip-coin". Распределение в виде таблицы не было использовано так как количество ключей может быть заранее неизвестно, поэтому либо в какой-то момент "случайность" станет периодичностью, что ухудшит среднюю сложность, либо придётся создавать таблицу большого размера, чтобы сохранить "случайность".

3. В операциях `Next`, `Put` и `Get` ключи и значения копируются для безопасности.
4. Генерация уровня реализована через добавление в поле `next` узла `head` нового элемента в конец.
5. Вероятность $p=0.5$ была выбрана по бенчмарку, в котором для различных p из диапазона $0.01-0.99$ определялось среднее время и потребление памяти при вставке и получении ключа. Оно было выбрано так как при нём наблюдалось наибольшая производительность в смысле времени на операцию и относительно небольшое потребление памяти. К тому же это значение является "типичным".

Хорошими значениями p по бенчмарку также являются все значения p из диапазона $0.2-0.6$

4. Контроль

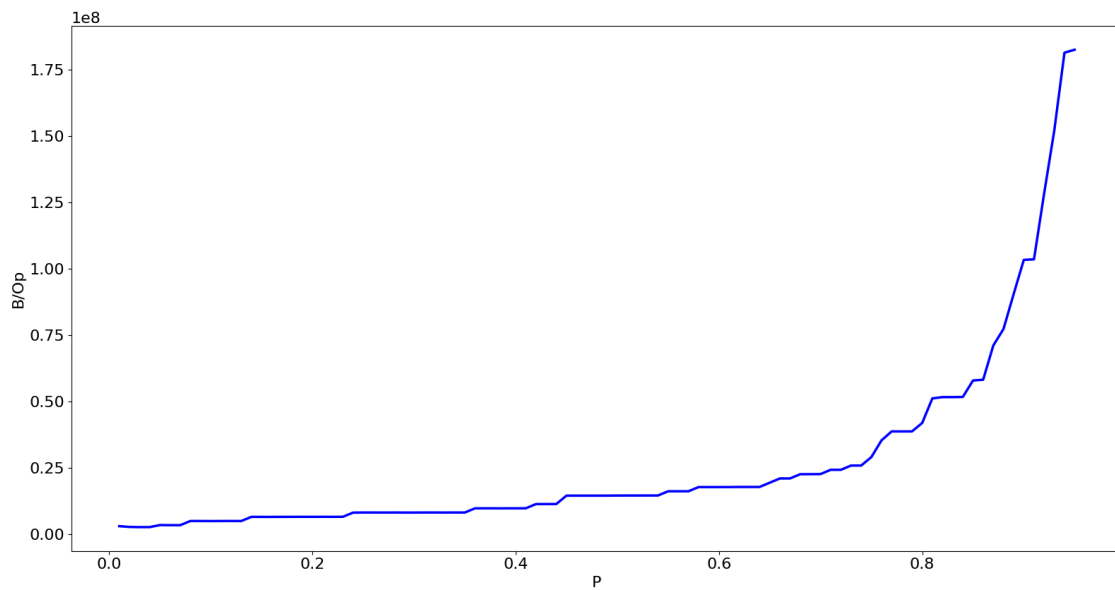
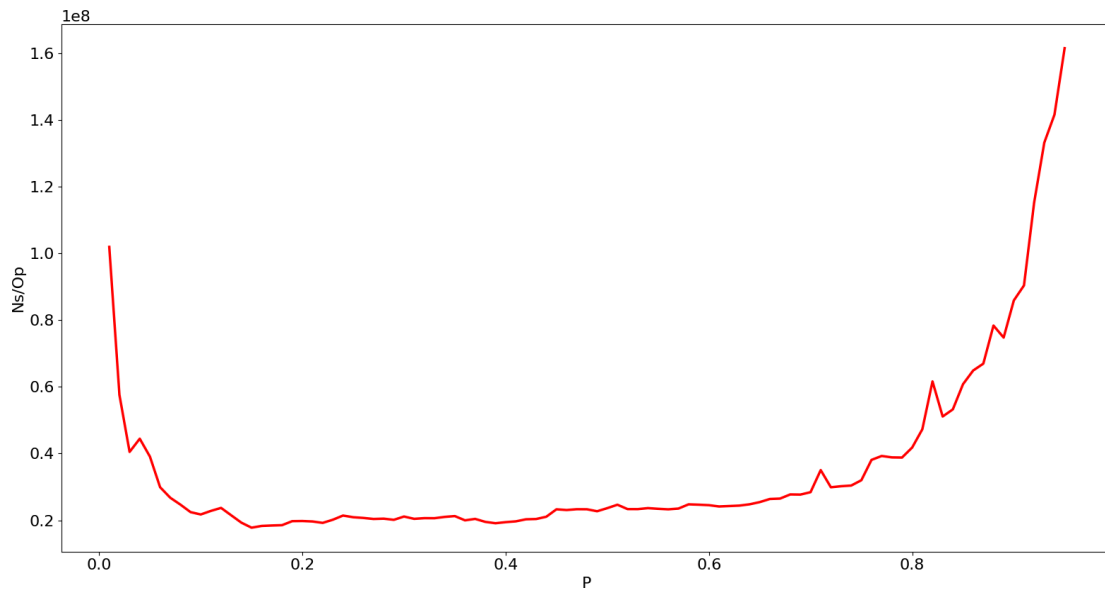
1. Операции `Put`, `Get`, `Delete` проверялись на тестах
 - `TestSkipList_BasicCRUD`
 - `TestSkipList_BasicOperations`
 - `TestSkipList_UnknownKey`
2. Операция `Scan` и `Next` (у итератора) проверялись на тестах
 - `TestSkipList_ScanOrderAndRange`
 - `TestSkipList_ScanEmpty`
 - `TestSkipList_ScanSingle`
 - `TestSkipList_ScanMultiple`
3. Для определения оптимальной вероятности использовался бенчмарк `BenchmarkPutGetProbability`, в котором для разных вероятностей p 100000 раз происходит обращение (`Get` или `Put`) в `SkipList`, в котором 100000 записей.
4. Бенчмарки `BenchmarkPutGetN` и `BenchmarkDeleteN` использовались для определения зависимости между потреблением памяти и средним временем операции в зависимости от количества элементов в `SkipList`.

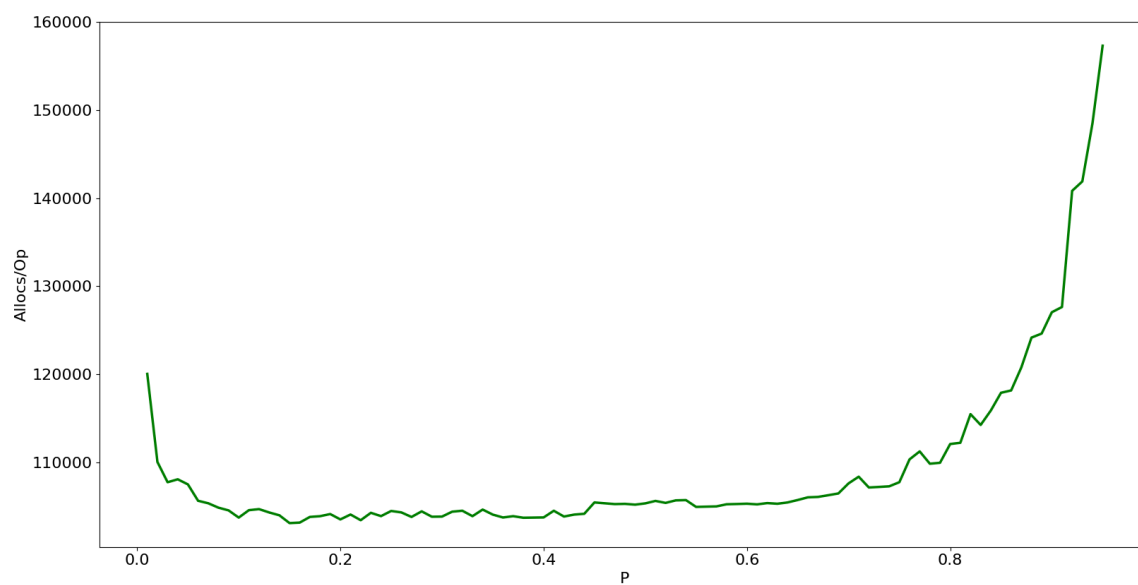
5. Известные ограничения

- Отсутствие поддержки конкурентного доступа к структуре данных
- Отсутствие логов изменений структуры
- Отсутствие снимков (снапшотов) структуры
- Отсутствие ограничения числа уровней
- Хранение только в оперативной памяти
- Отсутствие сетевого слоя

- Отсутствие возможности передать вероятность p при создании структуры (продиктовано реализуемым интерфейсом в коде)

6. Графики бенчмарка при разных p





7. Графики бенчмарка при разном размере структуры

